Lecture 11-1 · OOP(객체지향) 예제 01

강의 · Notion 아카이브

Public and Private

- Python은 **Object(객체)**지향 **Encapsulation(캡슐화)** 개념을 도입하여 **Class(클래스)** 명령어를 사용하나 기본적으로 데이터는 공개(public)함
- 데이터, 메소드 이름 앞에 (__)을 붙여 private로 선언할 수 있음 ex> __data, __method()

example

```
class Student:
    def __init__(self, name, age):
        self.__name = name
        self.__age = age

    def __show_name(self):
        print(self.__name)

    def __show_age(self):
        print(self.__age)

student = Student('Alice', 20)

print(student.__name) # error
print(student.__age) # error
student.__show_name() # error
student.__show_age() # error
```

```
AttributeError: 'Student' object has no attribute '__name'
```

- 메소드를 마음대로 호출할 수 없게 됨.

```
class Student:
    def __init__(self, name, age):
        self.__name = name
        self.__age = age

    def show_name(self):
        print(self.__name)
```

```
  def show_age(self):
      print(self.__age)

student = Student('Alice', 20)
student.show_name()
student.show_age()
```

```
Alice
20
```

- 반면 같은 구성에서 (__)을 없애면 잘 작동함.

Class example (**Stack(스택)**)

- Stack class를 정의하고, push, pop, peek, is empty 메소드를 구현.
- > Stack 클래스를 정의하고, push, pop, peek, is_empty 메소드를 구현
  - push : 스택에 데이터를 추가, 파라미터로 전달된 item을 items **list(리스트)**에 추가
  - pop : 스택에서 데이터를 꺼내는 메소드, 스택이 비어있을 경우 None 값을 반환, 스택이 비어있지 않을 경우 가장 마지막에 추가된 데이터를 꺼내서 반환.
  - peek : 스택의 맨 위에 있는 데이터를 반환하는 메소드, 스택이 비어있을 경우 None 값을 반환
  - is_empty : 스택이 비어있는지 확인하는 메소드, 스택이 비어있으면 True 값을, 그렇지 않으면 False 값을 반환

```
class Stack:
  def __init__(self, *values):
      self.Stack_list = []
      for value in values:
          self.Stack_list.append(value)

  def push(self, value):
      self.Stack_list.append(value)

  def pop(self):
      if self.Stack_list == []:
          return None
      return self.Stack_list.pop()

  def peek(self):
      if self.Stack_list == []:
          return None
      return self.Stack_list[-1]
```

```
    def is_empty(self):
        return self.Stack_list == []
```

```
stack = Stack()
stack.push(1)
stack.push(2)
stack.push(3)

print(stack.peek())
print(stack.pop())
print(stack.pop())
print(stack.pop())
print(stack.pop())
```

```
3
3
2
1
None
```

Inheritance(상속)example (Method(메서드)overriding)

- > Student 클래스 : 부모 클래스로 정의하고, Undergraduate 클래스와 Graduate 클래스가 Student 클래스를 상속 받아 생성
  - Student 클래스 : 이름(name)과 성적(grade)을 **Instance variable(인스턴스 변수)**로 가지며, display 메소드를 구현, display 메소드는 **Instance(인스턴스)** 변수의 값을 출력
  - Undergraduate 클래스 : Student 클래스를 상속받아 생성, 전공(major)이 추가된 클래스
  - init 메소드 : super() **Function(함수)**를 이용하여 상위 클래스의 **Constructor(생성자)**를 호출하고, 전공(major)을 추가로 초기화
  - display 메소드 : 상위 클래스의 display 메소드를 호출하고, 전공(major) 정보를 출력
  - Graduate 클래스 : Student 클래스를 상속받아 생성, 지도 교수(advisor)가 추가된 클래스
  - init 메소드에서 super() 함수를 이용하여 상위 클래스의 생성자를 호출하고, 지도 교수(advisor)를 추가로 초기화
  - display 메소드 : 마찬가지로 상위 클래스의 display 메소드를 호출하고, 지도 교수(advisor) 정보를 출력

```
class Student:

    def __init__(self, name, grade):
        self.name = name
        self.grade = grade
```

```
    def display(self):
        print(f"name = {self.name}")
        print(f"grade = {self.grade}")

class Undergraduate(Student):

    def __init__(self, name, grade, major):
        super().__init__(name, grade)
        self.major = major

    def display(self):
        super().display()
        print(f"major = {self.major}")

class Graduate(Student):

    def __init__(self, name, grade, advisor):
        super().__init__(name, grade)
        self.advisor = advisor

    def display(self):
        super().display()
        print(f"advisor = {self.advisor}")
```

```
student = Student("김학생", 90)
student.display()

undergrad = Undergraduate("이학생", 85, "컴퓨터공학")
undergrad.display()

grad = Graduate("박학생", 95, "김교수")
grad.display()
```

```
name = 김학생
grade = 90
name = 이학생
grade = 85
major = 컴퓨터공학
name = 박학생
```

```
grade = 95
advisor = 김교수
```

Operator Overloading(연산자 오버로딩)example

- > **__radd__** : + 연산자의 왼쪽에 있는 피연산자가 MyNumber 객체가 아닐 때도 처리, **__radd__** 메서드에서는 self와 other 두 개의 인자 받음. self는 현재 객체이며, other는 더할 대상 객체, **__radd__** 메서드에서는 other와 self.value를 더한 결과를 새로운 MyNumber 객체로 반환
  - num1 + num2 연산을 수행하면, num1.**__add__**(num2)가 호출되어 MyNumber(10 + 20)이 출력
  - 5 + num1 연산을 수행하면, num1.**__radd__**(5)가 호출되어 MyNumber(5 + 10)이 출력
  - **__add__** 메서드와 **__radd__** 메서드를 함께 사용하여, + 연산자의 양쪽 피연산자가 모두 MyNumber 객체이거나, 왼쪽 피연산자가 정수인 경우에도 처리
- __radd__는 “right add”, 즉 오른쪽 피연산자일 때의 덧셈 연산자 오버로딩을 처리하는 특별 메소드

```
class MyNumber:
    def __init__(self, value):
        self.value = value

    def __repr__(self):
        return str(self.value) # value 값을 문자열로 변환하여 반환

    def __add__(self, other):
        if isinstance(other, MyNumber):
            return MyNumber(self.value + other.value)
        else:
            return MyNumber(self.value + other)

    def __radd__(self, other):
        return MyNumber(self.value + other)

    def __iadd__(self, other): # '+=' 연산자 재정의
        if isinstance(other, MyNumber):
            self.value += other.value
        else:
            self.value += other
        return self
```

Polymorphism(다형성)example

```
class Shape:
    def area(self):
```

```
        pass
    def draw(self):
        pass

class Rectangle(Shape):
    def __init__(self, width, height):
        self.width = width
        self.height = height
    def area(self):
        return self.width * self.height
    def draw(self):
        print(f"Drawing a {self.width}x{self.height} rectangle.")

class Triangle(Shape):
    def __init__(self, base, height):
        self.base = base
        self.height = height
    def area(self):
        return 0.5 * self.base * self.height
    def draw(self):
        print(f"Drawing a triangle with base {self.base} and height {self.height}.")
```

← 프로그래밍과 문제해결 · CSED101 목차로